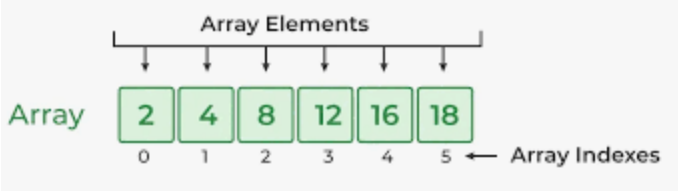
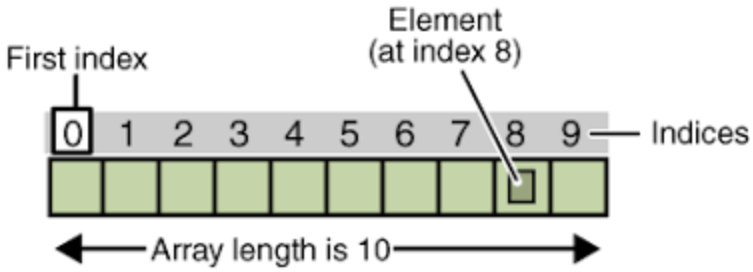
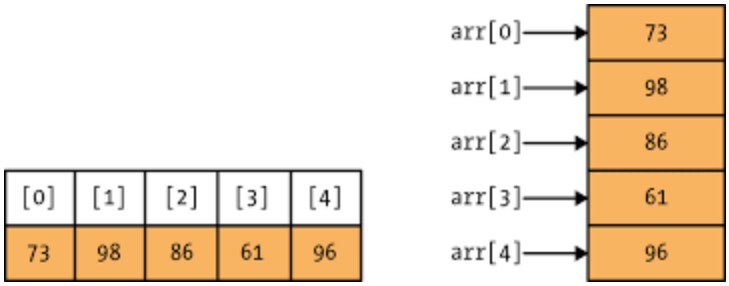
Arrays

There exists a special data structure named `Array`, to store ordered collections.



An array in javascript is a resizable list of containers. Each container can store any value assigned to it. A variable is a single container that can store a value. An array is like a list of variables. Each container is called an element. Each element is identified by its index number. The first index is always 0 and the last is always the length of the array -1.





Declaration

There are two syntaxes for creating an empty array:

```
let arr = new Array();
let arr = [];
```

Almost all the time, the second syntax is used. We can supply initial elements in the brackets:

```
let fruits = ["Apple", "Orange", "Plum"];
```

Array elements are numbered, starting with zero.

We can get an element by its number in square brackets:

```
let fruits = ["Apple", "Orange", "Plum"];

alert( fruits[0] ); // Apple
alert( fruits[1] ); // Orange
alert( fruits[2] ); // Plum
```

We can replace an element:

```
fruits[2] = 'Pear'; // now ["Apple", "Orange", "Pear"]
```

...Or add a new one to the array:

```
fruits[3] = 'Lemon'; // now ["Apple", "Orange", "Pear", "Lemon"]
```

The total count of the elements in the array is its `length`:

```
let fruits = ["Apple", "Orange", "Plum"];

alert( fruits.length ); // 3
```

We can also use `alert` to show the whole array.

```
let fruits = ["Apple", "Orange", "Plum"];

alert( fruits ); // Apple,Orange,Plum
```

An array can store elements of any type.

Methods pop/push, shift/unshift

A queue is one of most common uses of an array. In computer science, this means an ordered collection of elements which supports two operations:

- **`push`** appends an element to the end.
- **`shift`** get an element from the beginning, advancing the queue, so that the 2nd element becomes the 1st.



Arrays support both operations.

In practice we meet it very often. For example, a queue of messages that need to be shown on-screen.

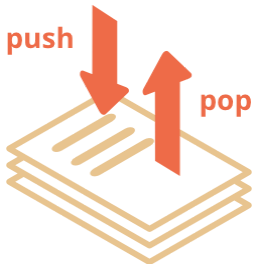
There's another use case for arrays – the data structure named stack.

It supports two operations:

- **`push`** adds an element to the end.
- **`pop`** takes an element at the end.

So new elements are added or taken always from the "end".

A stack is usually illustrated as a pack of cards: new cards are added to the top or taken from the top:



For stacks, the latest pushed item is received first, that's also called LIFO (Last-In-First-Out) principle. For queues, we have FIFO (First-In-First-Out).

Arrays in JavaScript can work both as a queue and as a stack. They allow to add/remove elements both to/from the beginning or the end.

Methods that work with the end of the array:

pop

Extracts the last element of the array and returns it:

```
let fruits = ["Apple", "Orange", "Pear"];

alert( fruits.pop() ); // remove "Pear" and alert it

alert( fruits ); // Apple, Orange
```

push

Append the element to the end of the array:

```
let fruits = ["Apple", "Orange"];

fruits.push("Pear");

alert( fruits ); // Apple, Orange, Pear
```

The call `fruits.push(...)` is equal to `fruits[fruits.length] = ...`.

Methods that work with the beginning of the array:

shift

Extracts the first element of the array and returns it:

```
let fruits = ["Apple", "Orange", "Pear"];

alert( fruits.shift() ); // remove Apple and alert it

alert( fruits ); // Orange, Pear
```

unshift

Add the element to the beginning of the array:

```
let fruits = ["Orange", "Pear"];

fruits.unshift('Apple');

alert( fruits ); // Apple, Orange, Pear
```

Methods `push` and `unshift` can add multiple elements at once:

```
let fruits = ["Apple"];

fruits.push("Orange", "Peach");
fruits.unshift("Pineapple", "Lemon");

// ["Pineapple", "Lemon", "Apple", "Orange", "Peach"]
alert( fruits );
```

Loops

One of the oldest ways to cycle array items is the `for` loop over indexes:

```
let arr = ["Apple", "Orange", "Pear"];

for (let i = 0; i < arr.length; i++) {
  alert( arr[i] );
}
```

Summary

Array is a special kind of objects, suited to store and manage ordered data items.

- The declaration:
- `// square brackets (usual)`
- `let arr = [item1, item2...];`
-
- The `length` property is the array length or, to be precise, its last numeric index plus one. It is auto-adjusted by array methods.
- `push(...items)` adds `items` to the end.
- `pop()` removes the element from the end and returns it.
- `shift()` removes the element from the beginning and returns it.
- `unshift(...items)` adds items to the beginning.

To loop over the elements of the array:

- `for(let i=0; i<arr.length; i++)` – works fastest, old-browser-compatible.

Questions and Exercise

1. What is this code going to show?

```
let fruits = ["Apples", "Pear", "Orange"];

// push a new value into the "copy"
let shoppingCart = fruits;
shoppingCart.push("Banana");

// what's in fruits?
alert( fruits.length ); // ?
```

2. Create an array `styles` with items “Jazz” and “Blues”.

Append “Rock-n-Roll” to the end.

Replace the value in the middle by “Classics”. Your code for finding the middle value should work for any arrays with odd length.

Strip off the first value of the array and show it.

Prepend `Rap` and `Reggie` to the array.

The array in the process:

```
Jazz, Blues
Jazz, Bues, Rock-n-Roll
Jazz, Classics, Rock-n-Roll
Classics, Rock-n-Roll
Rap, Reggie, Classics, Rock-n-Roll
```

3. Create a program that fills a 10 element array with 10 random numbers between 1 and 100. Display these numbers.

4. Create a program that fills a 100 element array with the squares of the numbers from 1 to 100. Display the index of each element alongside it’s value.
   Example:

Index	**Square of Index**
0	**0**
1	**1**
2	**4**
3	**9**
etc.	

5. Randomly fill a 1000 element array with random numbers between 1 and 100. Use the array to calculate the average of all those numbers.

6. Fill an array with 100 random numbers between 1 and 100. Ask a user for any number between 1 and 100. Find out if and where the number first appears in the array.

7. Do the same as #6 but this time displays how many instances of the number appear in the array.

8. Create an array of 10 different animals. Ask someone for the name of one of the animals and print out the index of the location where that animal is found in the list.

9. Using the same array from the last example, print the list of animals. Switch the first and last animals in the array and then print the list again.

10. Fill an array with 100 random numbers between 1 and 100. Write a program to display odd and even integers in an array.